



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第六章 集合与字典



- 集合及其表示
- 并查集与等价类
- 字典
- 散列



6.3 字典 (Dictionary)



6.3.1 字典的概念

- 字典：以集合为基础，支持Member、Insert、Remove运算的抽象数据类型
- 字典是一些元素的集合，每个元素有一个称作**关键码 (key)**的域，不同元素的关键码互不相同。
 - ✓ 文件 (File)：记录(record)
 - ✓ 表格 (Table)：表项(entry)
- 在讨论字典抽象数据类型时，把字典定义为<**名字-属性**>对的集合。
- 根据问题的不同，可以为名字和属性赋予不同的含义。



- 例如，在图书馆检索目录中，名字是书名，属性是索引号及作者等信息；在计算机活动文件表中，名字是文件名，属性是文件地址、大小等信息。
- 一般来说，有关字典的操作有如下几种：
 1. 确定一个指定的名字是否在字典中；
 2. 搜索出该名字的属性；
 3. 修改该名字的属性；
 4. 插入一个新的名字及其属性；
 5. 删除一个名字及其属性。



字典的抽象数据类型



```
const int DefaultSize = 26;
template <class Name, class Attribute>
class Dictionary {
//对象: 一组<名字-属性>对, 其中, 名字是唯一的
public:
    Dictionary (int size = DefaultSize);    //构造函数
    bool Member (Name name);
        //判name是否在字典中
    Attribute *Search (Name name);
        //在字典中搜索关键码与name匹配的表项
```



```
void Insert (Name name, Attribute attr);  
    //若name在字典中, 则修改相应<name, attr>对的  
    //attr项; 否则插入<name, attr>到字典中  
void Remove (Name name);  
    //若name在字典中, 则在字典中删除相应的  
    //<name, attr>对  
};
```

- 用文件的记录 (record) 或表格的表项 (entry) 来表示单个元素时, 用:
 (关键码key, 记录或表项位置指针adr)

构成搜索某一指定记录或表项的索引项。



6.3.2 字典的线性表描述



- 字典可以保存在线性序列 $(e_1, e_2, \dots)$ 中，其中 e_i 是字典中的元素，其关键码从左到右依次增大。为了适应这种描述方式，可以定义**有序顺序表**和**有序链表**。
- 用有序链表来表示字典时，链表中的每个结点表示字典中的一个元素，各个结点按照结点中保存的数据值**非递减**链接，即 $e_1 \leq e_2 \leq \dots$ 。因此，在一个有序链表中寻找一个指定元素时，一般不用搜索整个链表。



有序顺序表的类定义



```
#include <iostream.h>
#include "SeqList.h"
const int defaultSize = 50;
template <class K, class E>
class SortedList : public SeqList {
public:
    int Search (K k1) const;           //搜索
    void Insert (const K k1, E& e1);   //插入
    bool Remove (const K k1, E& e1);   //删除
};
```




基于有序顺序表的顺序搜索算法



```
template <class K, class E>
int SortedList<K, E>::Search (K k1) const {
    //顺序搜索关键码为k1的数据对象
    int n = last+1;
    for (int i = 1; i <= n; i++)
        if (data[i-1] == k1) return i;    //成功
        else if (data[i-1] > k1) break;
    return 0;    //顺序搜索失败, 返回失败信息
};
```

- 算法中的 “==”和 “>”都是重载函数，在定义K时定义它们的实现。



有序顺序表顺序搜索的时间代价



- 衡量一个搜索算法的时间效率的标准是：在搜索过程中关键码平均比较次数，也称为**平均搜索长度ASL** (Average Search Length)，通常它是字典中元素总数 n 的函数。
- 设搜索第 i 个元素的概率为 p_i ，搜索到第 i 个元素所需比较次数为 c_i ，则搜索成功的平均搜索长度：

$$ASL_{succ} = \sum_{i=1}^n p_i \cdot c_i \quad \left(\sum_{i=1}^n p_i = 1 \right)$$



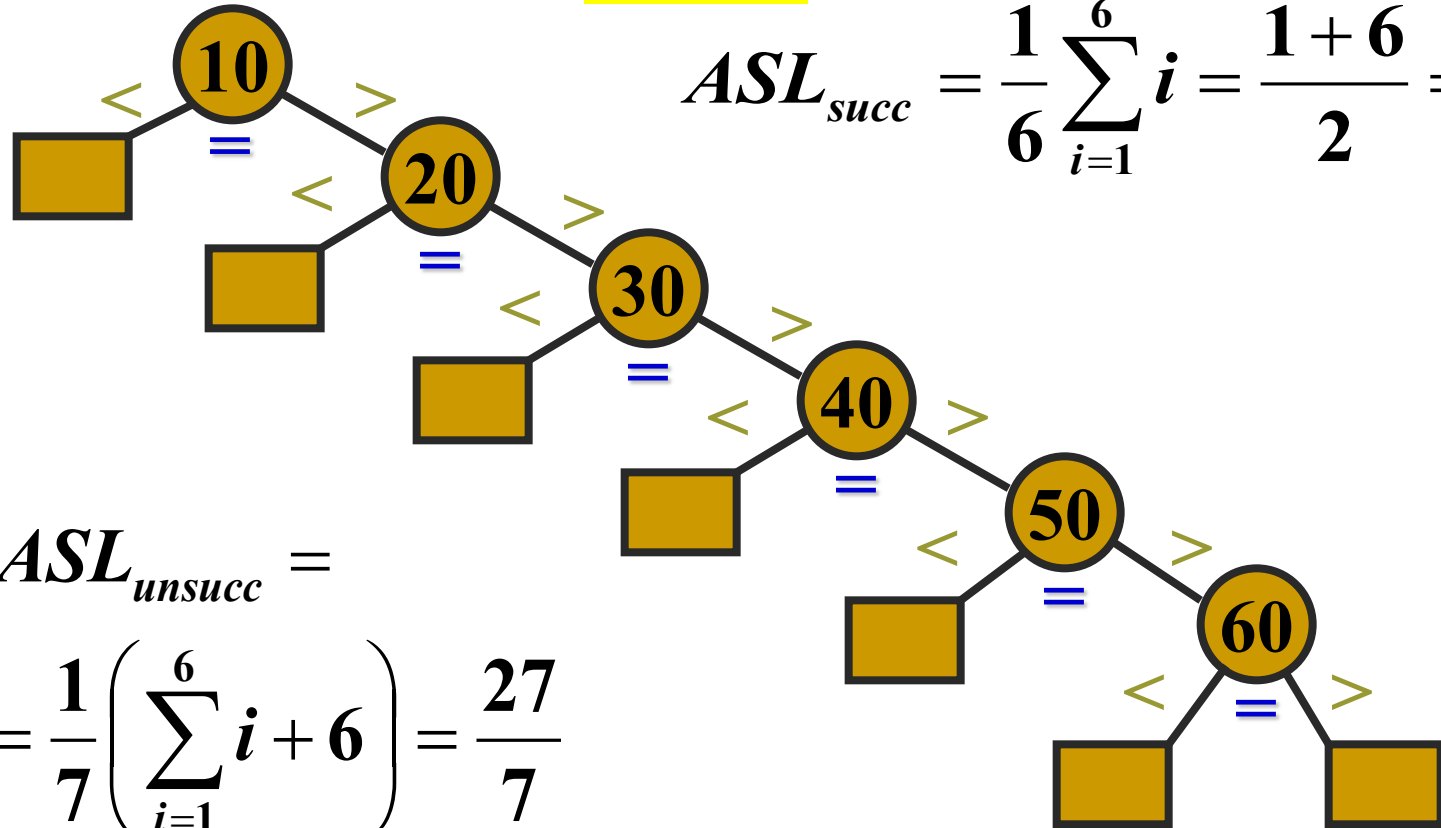
- 在顺序搜索情形，搜索第 i ($1 \leq i \leq n$) 个元素需要比较 i 次，假定按**等概率搜索**各元素：

$$ASL_{succ} = \sum_{i=1}^n p_i \times c_i = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \frac{(n+1)n}{2} = \frac{n+1}{2}$$

- 这与一般顺序表情形相同。但搜索不成功时不需一直比较到表尾，只要发现下一个元素的值比给定值大，就可断定搜索不成功。
- 设一个有 n 个表项的表，查找失败的位置有 $n+1$ 个，可以用判定树加以描述。搜索成功时停在内结点，搜索失败时停在外结点。



- 例如，有序顺序表 (10, 20, 30, 40, 50, 60) 的顺序搜索的分析 (使用判定树)



$$ASL_{succ} = \frac{1}{6} \sum_{i=1}^6 i = \frac{1+6}{2} = \frac{7}{2}$$

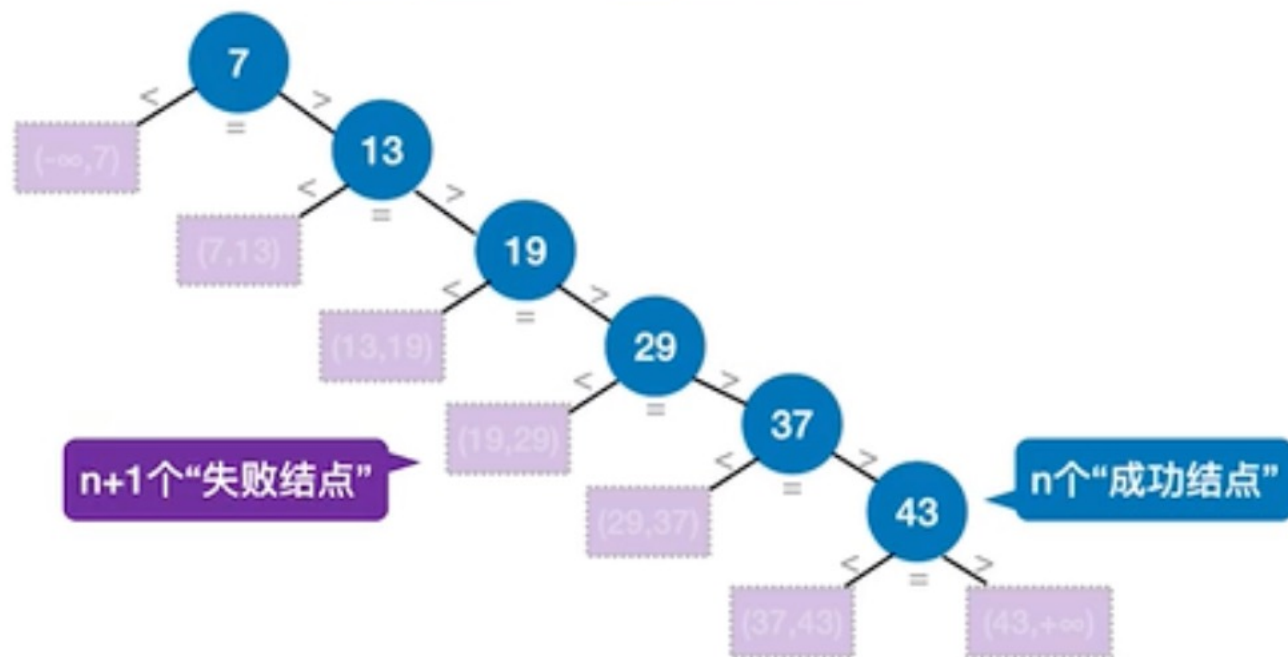
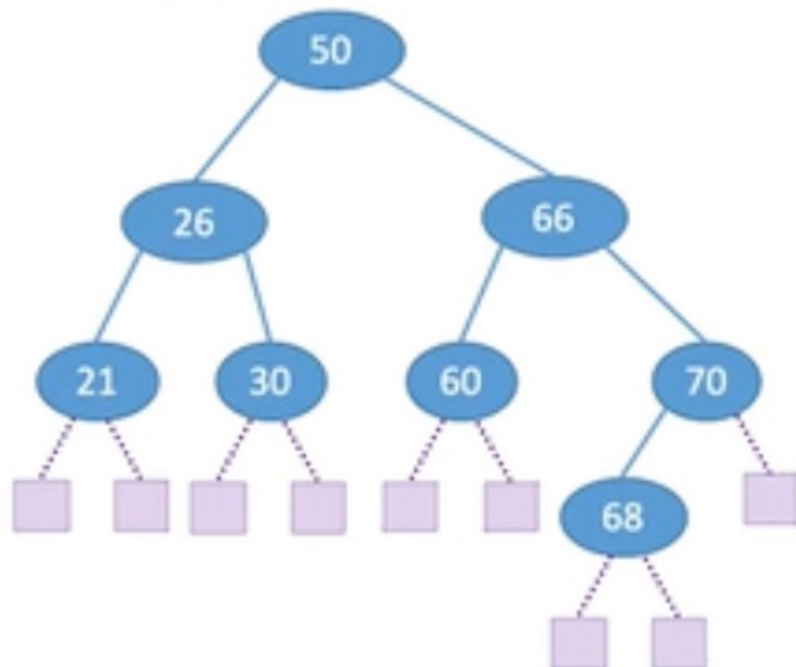
$$ASL_{unsucc} = \frac{1}{7} \left(\sum_{i=1}^6 i + 6 \right) = \frac{27}{7}$$



- 判定树是一种扩充二叉树。
- 内结点代表顺序表中已有的元素；
- 外结点代表失败结点，表示在两个相邻已有元素值之间的值。假定表中所有失败位置的搜索概率相同，则搜索不成功的平均搜索长度：

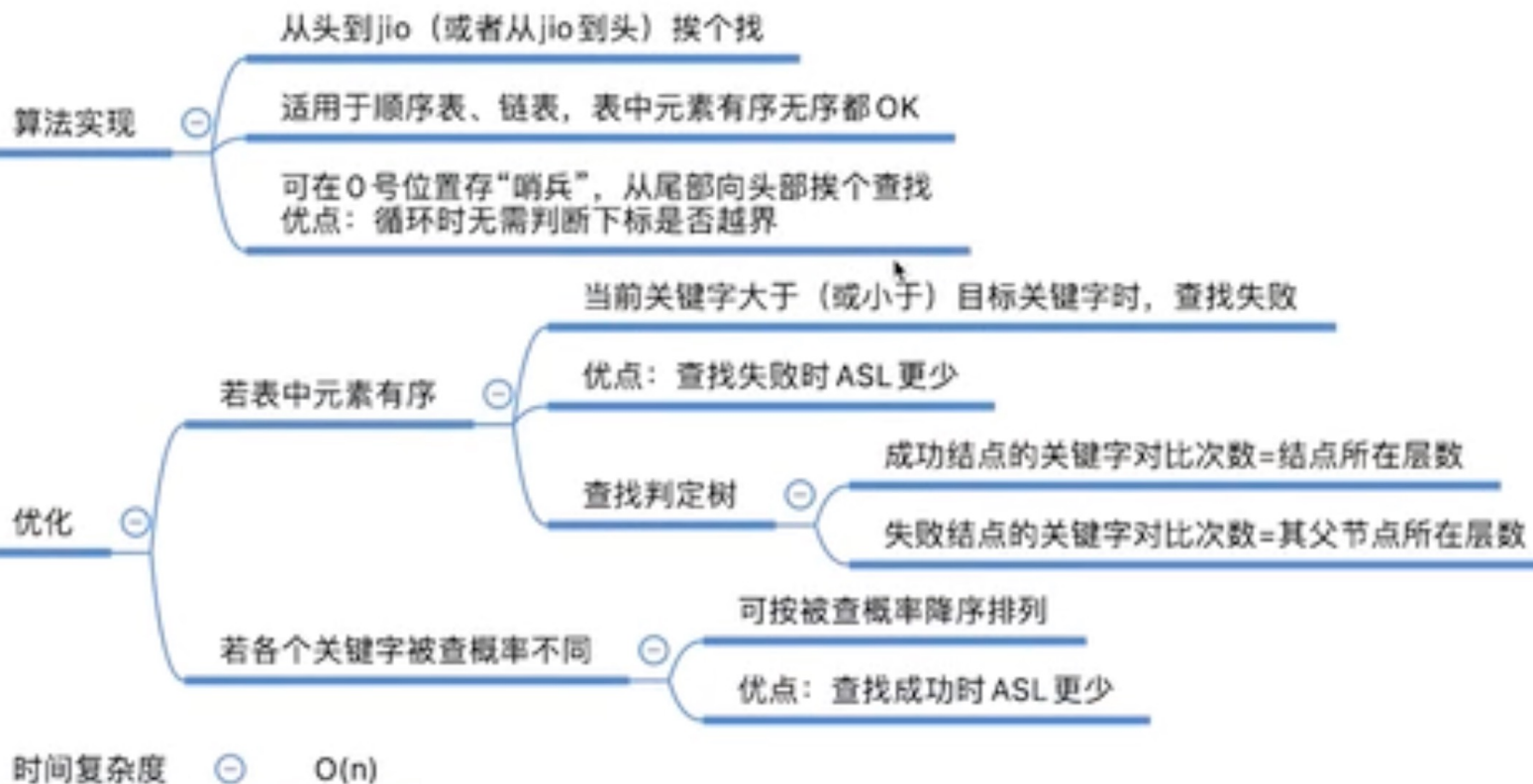
$$ASL_{unsucc} = \frac{1}{n+1} \left(\sum_{i=1}^n i + n \right)$$

- 时间代价为 $O(n)$ 。
- 为了加速搜索，在有序顺序表的情形，可以采用折半搜索，也称二分搜索，时间代价可减小到 $O(\log_2 n)$ 。





顺序查找

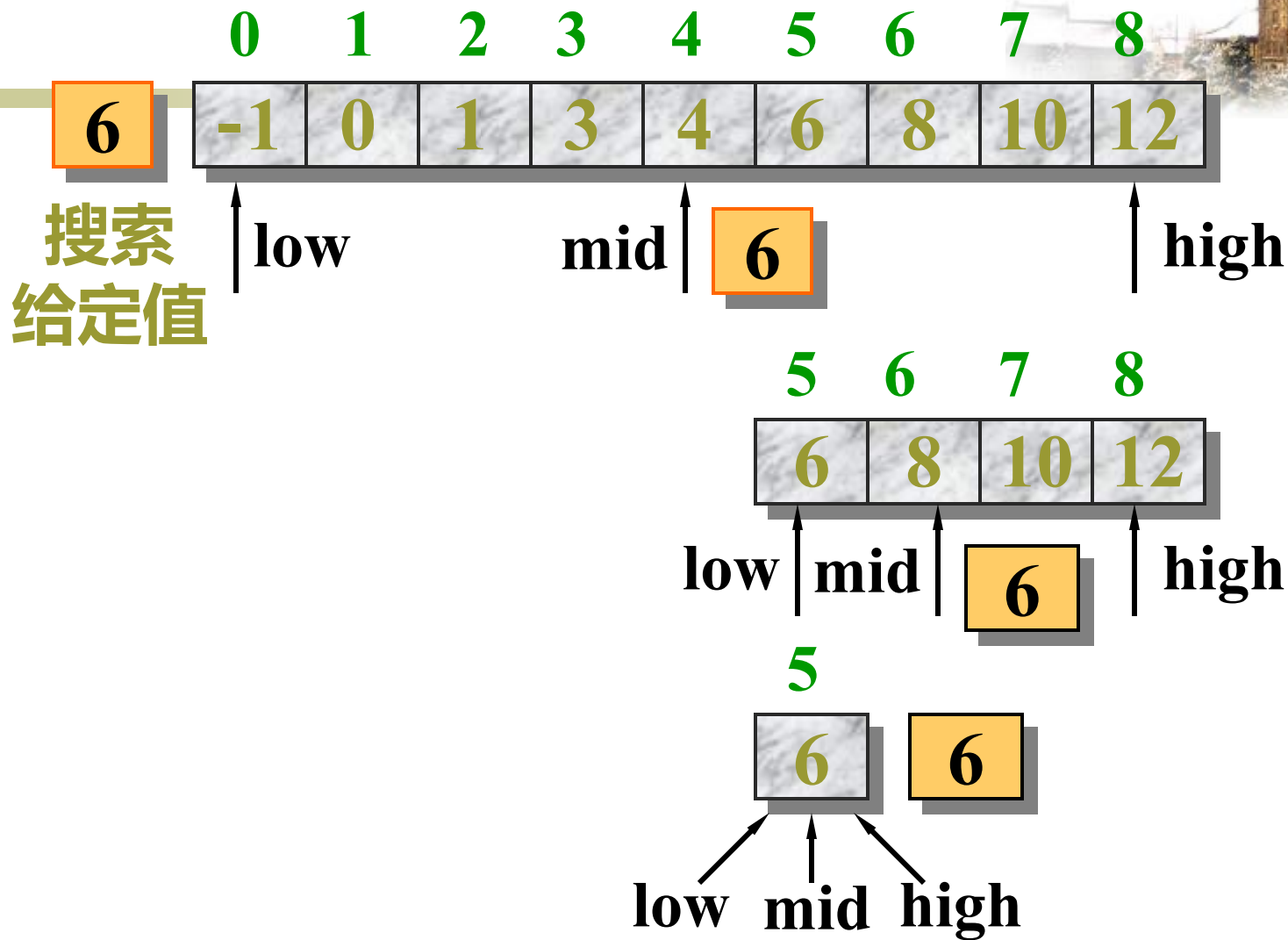




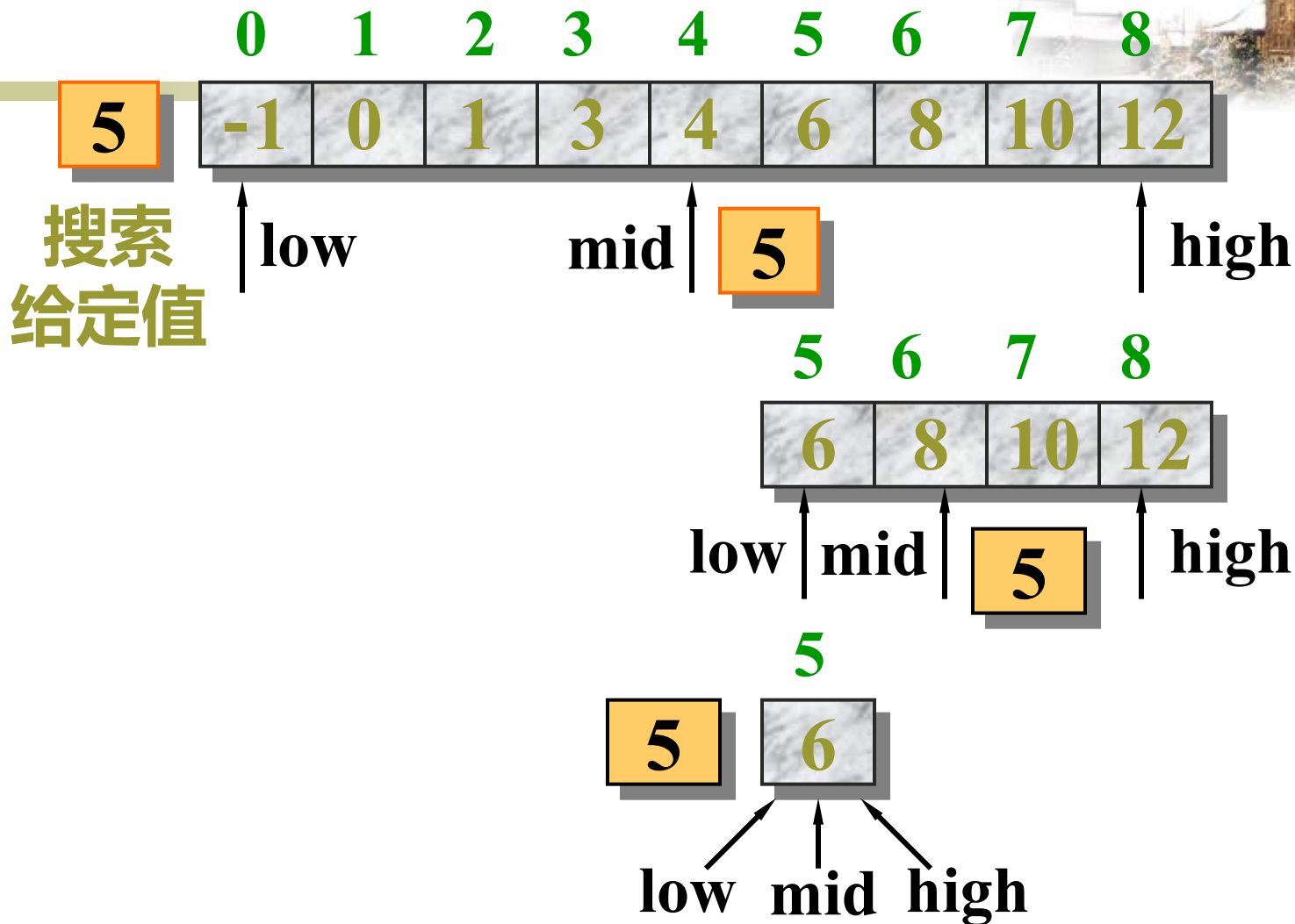
基于有序顺序表的折半搜索



- 设 n 个元素存放在一个有序顺序表中。
- 折半搜索时，先求位于搜索区间正中的元素的下标 mid ，用其关键码与给定值 x 比较：
 - ◆ $data[mid].key == x$ ，搜索成功；
 - ◆ $data[mid].key > x$ ，把搜索区间缩小到表的前半部分，继续折半搜索；
 - ◆ $data[mid].key < x$ ，把搜索区间缩小到表的后半部分，继续折半搜索。
- 如果搜索区间已缩小到一个对象，仍未找到想要搜索的对象，则搜索失败。



搜索成功的例子



搜索失败的例子



```
template<class K, class E>
int SortedList<K, E>::BinarySearch
    (K k1, const int low, const int high ) const {
//折半搜索的递归算法, 用到E的重载操作 "<"和 ">"
    int mid = 0;                                //元素序号从0开始
    if (low <= high) {
        mid = (low + high) / 2;
        if (data[mid] < k1)
            mid = BinarySearch (k1, mid +1, high);
        else if (data[mid] > k1)
            mid = BinarySearch (k1, low, mid -1);
        else return mid;
    }
    return 0;
};
```



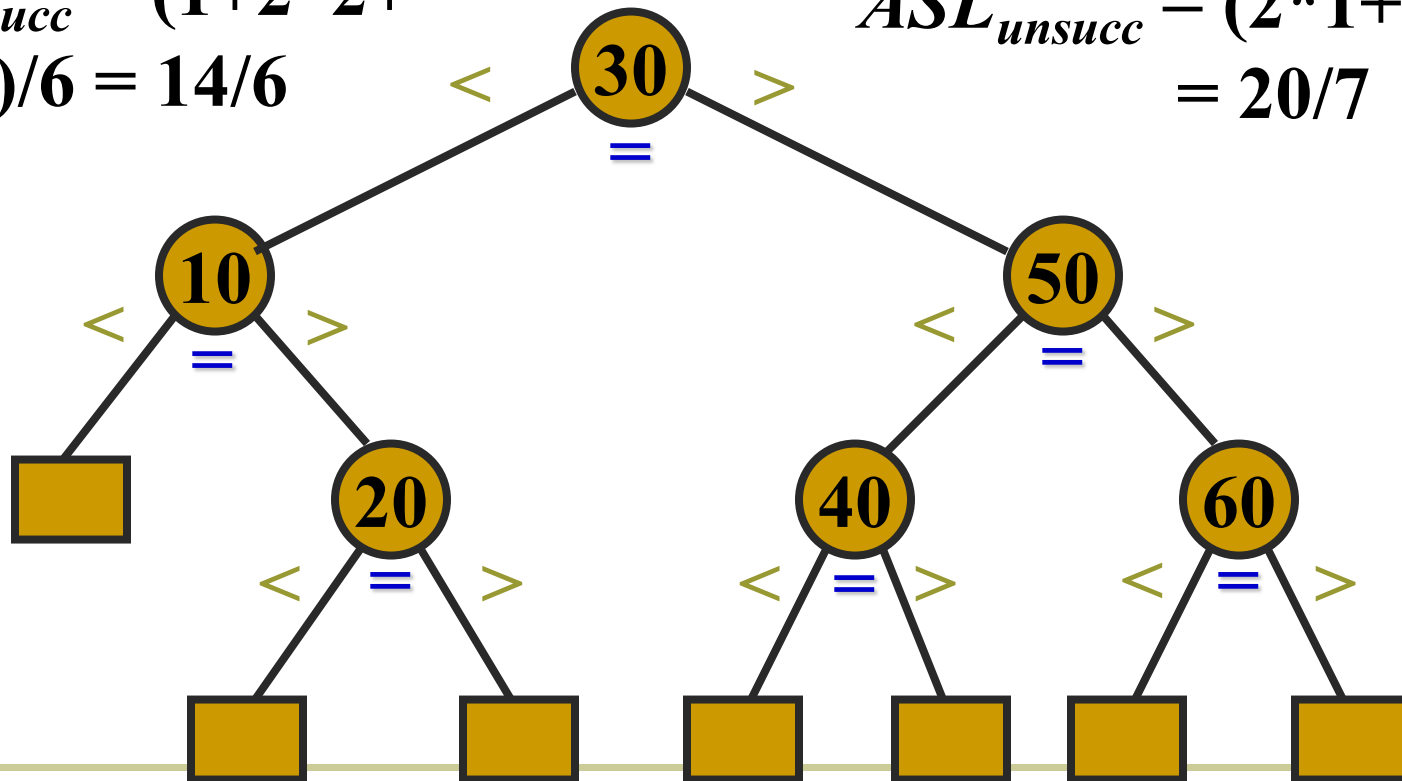
```
template<class K, class E>
int SortedList <K, E>::BinarySearch (K k1) const {
//折半搜索的迭代算法，用到K的重载操作 "<"和 ">"
    int high = n-1, low = 0, mid; //元素序号从0开始
    while (low <= high) {
        mid = (low + high) / 2;
        if (data[mid] < k1) low = mid+1;
                                //右缩搜索区间
        else if (data[mid] > k1) high = mid-1;
                                //左缩搜索区间
        else return mid;
                                //搜索成功
    }
    return 0;
                                //搜索失败
}
```

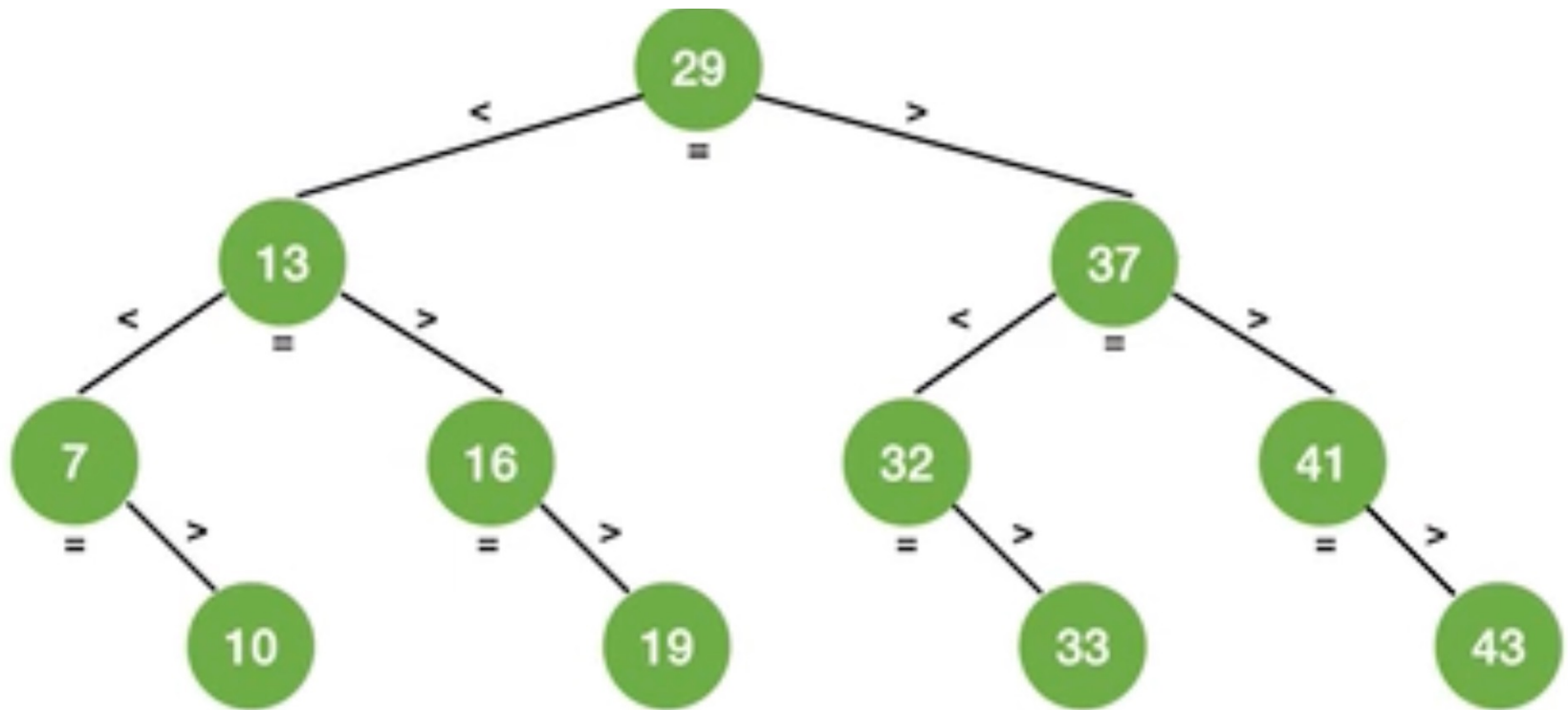


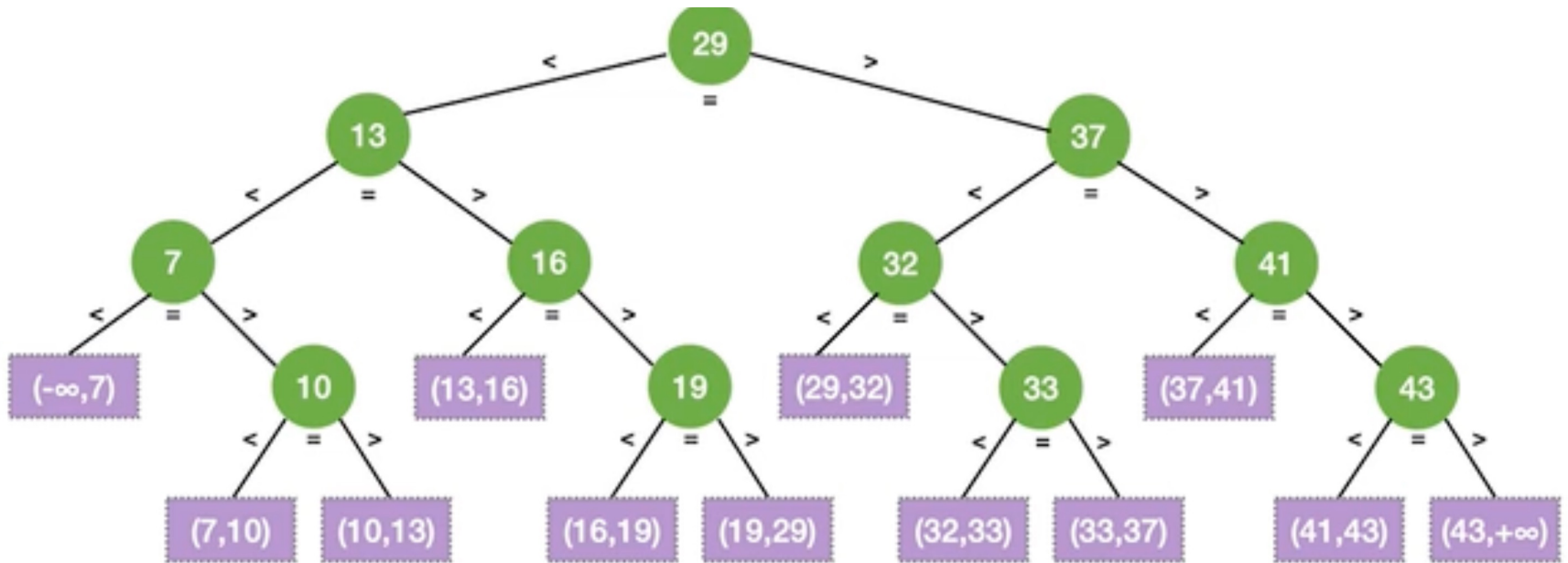
- 分析有序顺序表 (10, 20, 30, 40, 50, 60) 的折半搜索算法性能的判定树:

$$ASL_{succ} = (1 + 2 * 2 + 3 * 3) / 6 = 14/6$$

$$ASL_{unsucc} = (2 * 1 + 3 * 6) / 7 = 20/7$$



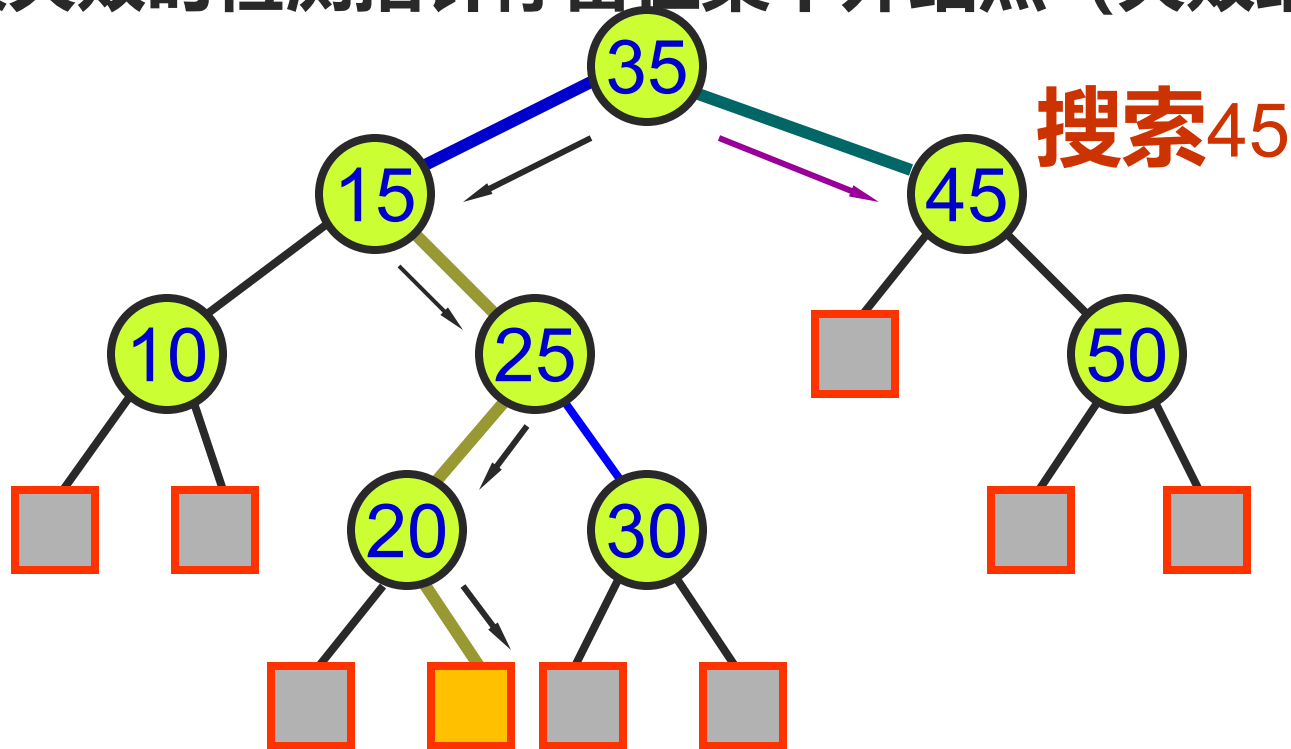






判定树也是扩充二叉树

- 搜索成功时检测指针停留在树中某个内结点上。
- 搜索失败时检测指针停留在某个外结点（失败结点）上。



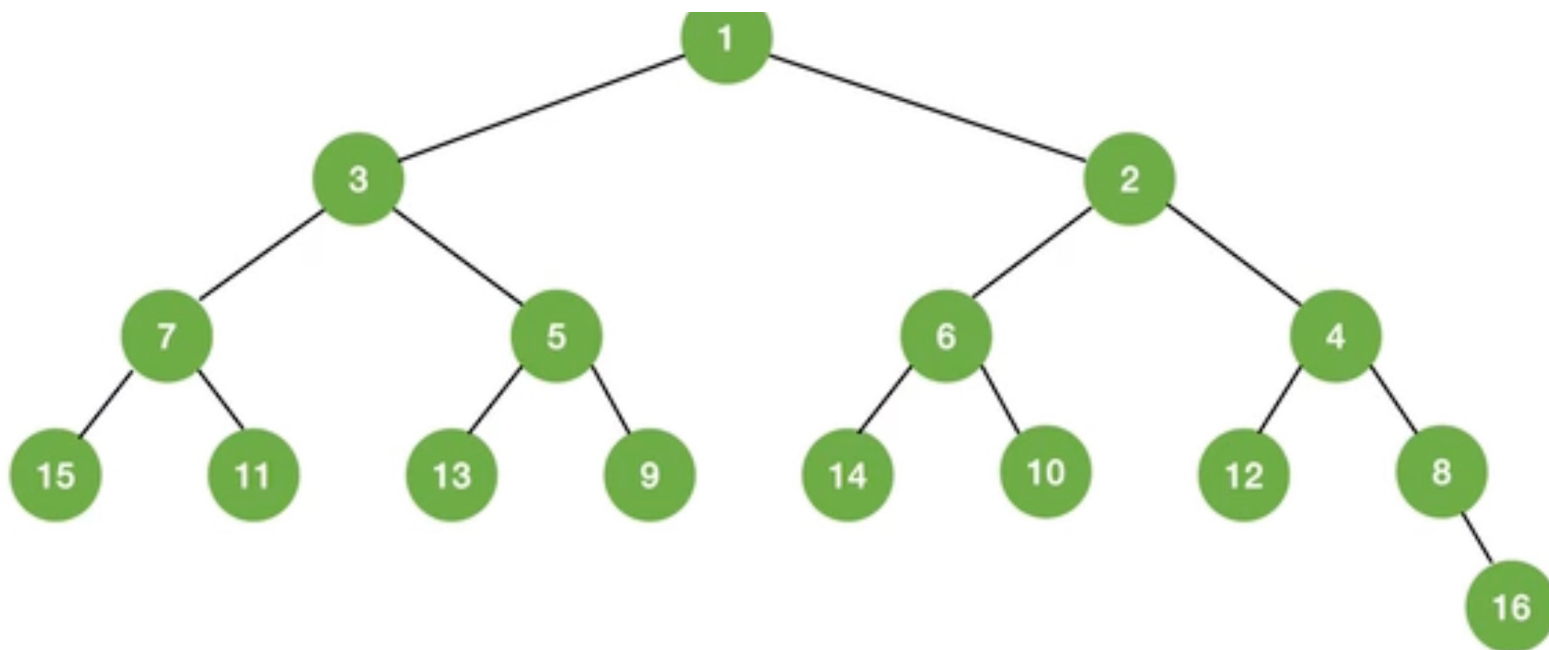
搜索22



构造折半查找判定树

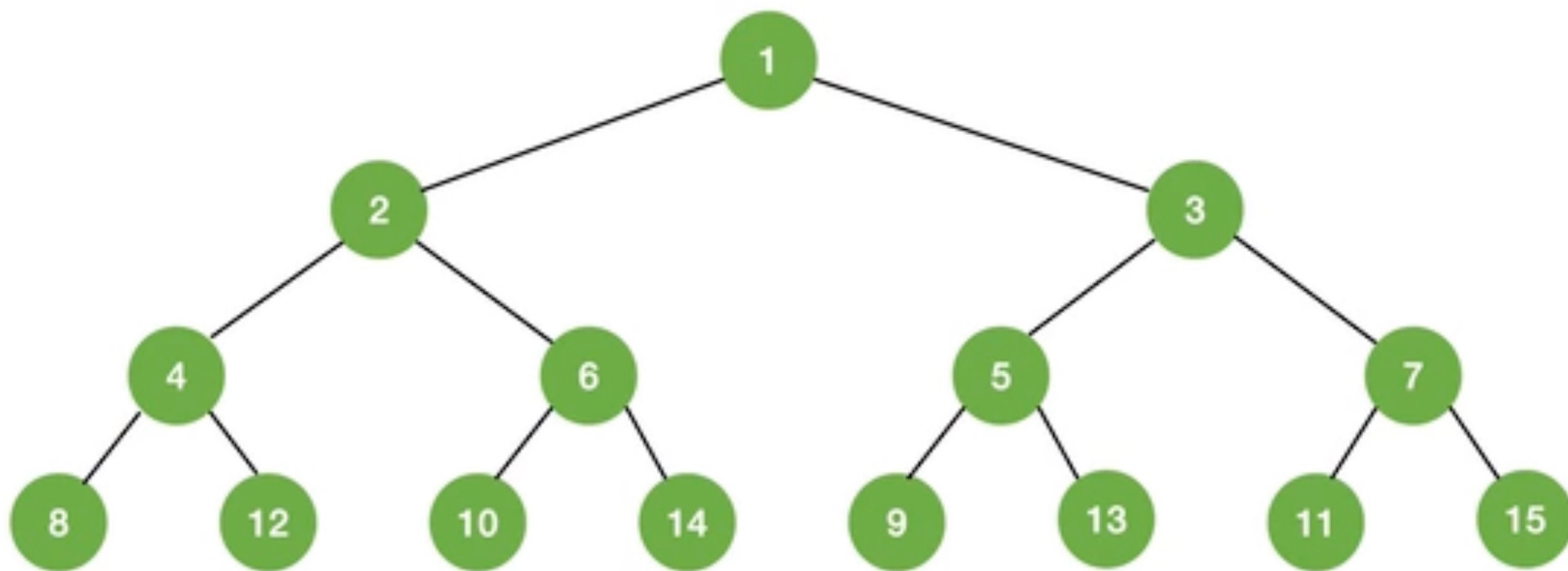


- 16个结点（mid向下取整）





折半查找的判定树中，若 $mid = \lfloor (low + high)/2 \rfloor$ ，则对于任何一个结点，必有：
左子树结点数-右子树结点数=0或1





折半搜索算法的性能分析



- 若设 $n = 2^h - 1$, 则描述折半搜索的判定树是高度为 h 的满二叉树 (加上失败结点高度为 $h + 1$)。

$$2^h = n + 1, h = \log_2(n + 1)$$

- 第1层结点有1个, 搜索第1层结点要比较1次;
第2层结点有2个, 搜索第2层结点要比较2次; ..., 第 i ($1 \leq i \leq h$) 层结点有 2^{i-1} 个, 搜索第 i 层结点要比较 2^{i-1} 次, ...。
- 假定每个结点的搜索概率相等, 即 $p_i = 1/n$, 则搜索成功的平均搜索长度为



$$ASL_{succ} = \frac{1}{n} (1 \times 2^0 + 2 \times 2^1 + 3 \times 2^2 + \cdots + h \times 2^{h-1})$$

可以用归纳法证明

$$\begin{aligned} & 1 \times 2^0 + 2 \times 2^1 + 3 \times 2^2 + \cdots + (h-1) \times 2^{h-2} + h \times 2^{h-1} \\ &= (h-1) \times 2^h + 1 \end{aligned}$$

这样，由 $2^h = n+1$, $h = \log_2(n+1)$

$$\begin{aligned} ASL_{succ} &= \frac{1}{n} ((h-1) \times 2^h + 1) = \frac{1}{n} ((n+1) \log_2(n+1) - n) \\ &= \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1 \end{aligned}$$



折半查找

适用范围

只适用于有序的顺序表

算法思想

在 $[low, high]$ 之间找目标关键字，每次检查 $mid = (low + high) / 2$

根据 mid 所指元素与目标关键字的大小调整 low 或 $high$ ，不断缩小 low 和 $high$ 的范围

若 $low > high$ 则查找失败

判定树

构造

由 mid 所指元素将原有元素分割到左右子树中

Key: 右子树结点数 - 左子树结点数 = 0 或 1

特性

折半查找的判定树是平衡的二叉排序树 (左 < 中 < 右)

折半查找判定树，只有最下面一层是不满的

若查找表有 n 个关键字，则失败结点有 $n+1$ 个

树高 $h = \lceil \log_2(n+1) \rceil$

(不包含失败结点)

时间复杂度

$O(\log_2 n)$



有序链表的类定义



```
#include <assert.h>
```

```
template <class K, class E>
```

```
struct ChainNode {                                //链表结点类定义
```

```
    K key;
```

```
    E attr;
```

```
    ChainNode<K, E> *link;
```

```
    ChainNode() : link (NULL) { };                //构造函数
```

```
    ChainNode (K k1, E& e1,                        //构造函数
```

```
                ChainNode<K,E> *next = NULL)
```

```
                : key (k1), attr (e1), link (next) { };
```

```
};
```



```
template <class K, class E>
```

```
class SortedChain {
```

//有序链表类定义

```
public:
```

```
    SortedChain () {
```

//构造函数

```
        first = new ChainNode<K, E>;
```

```
        assert (first != NULL);
```

```
};
```

```
~SortedChain ();
```

//析构函数

```
ChainNode<K, E> *Search (K k1);
```

//搜索

```
void Insert (const K k1, E& e1);
```

//插入

```
bool Remove (const K k1, E& e1);
```

//删除



```
ChainNode<K, E> *Begin () { return first->link; }  
                        //定位第一个
```

```
ChainNode<K, E> *Next (ChainNode<K, E>  
    *current) const {      //定位下一个  
    if (current != NULL) return current->link;  
    else return NULL;  
}
```

private:

```
    ChainNode<K, E> *first;    //链表的头指针  
};
```




搜索、插入与删除算法



```
template <class K, class E>
```

```
ChainNode<K, E> *SortedChain<T>::
```

```
Search (K k1) const {
```

```
    ChainNode<K, E> *p = first->link;
```

```
    while (p != NULL && p->key< k1)
```

```
        p = p->link;                //重载：元素判小于
```

```
    if ( p != NULL && p->key== k1) return p;
```

```
        //重载：元素判等于
```

```
    else return NULL;
```

```
};
```



```
template <class K, class E>
```

```
void SortedChain<K, E>::
```

```
Insert (const K k1, E& e1) {
```

```
    ChainNode<K, E> *p = first->link, *pre = first;
```

```
    ChainNode<K, E> *newNode;
```

```
    while (p != NULL && p->key<= k1)
```

//重载：元素判小于等于

```
        { pre = p; p = p->link; }    //寻找插入位置
```

```
    if (p != NULL && p->key== k1) p->attr= e1;
```

```
    else {
```

```
        newNode = new ChainNode<K, E>(k1, e1);
```

```
        if (newNode == NULL) {
```



```
    cerr << “存储分配失败! ” << endl;  
    exit (1);  
}  
newNode->link = p;  pre->link = newNode;  
}  
};
```



```
template <class K, class E>
```

```
bool SortedChain<K, E>::Remove (const K k1, E& e1) {
```

```
    ChainNode<K, E> *p = first->link, *pre = first;
```

```
    while (p != NULL && p->key < k1)
```

```
        { pre = p; p = p->link; }           //寻找删除位置
```

```
    if (p != NULL && p->key == k1) {
```

```
        //重载：元素关键码判等于
```

```
        pre->link = p->link;
```

```
        e1 = p->attr;
```

```
        delete p;
```

```
        return true;
```

```
    }
```

```
    else return false;
```

```
    //未找到删除结点
```

```
};
```



- 1. 字典数据结构的主要特点是:
 - A) 元素按插入顺序存储
 - B) 通过数值索引快速访问
 - C) 通过键 (**key**) 快速访问值 (**value**)
 - D) 元素自动排序
- 2. 当向字典中插入一个已存在的键时, 会发生:
 - A) 抛出异常
 - B) 忽略新值, 保留旧值
 - C) 用新值覆盖旧值
 - D) 同时保留新旧值
- 3. 字典顺序查找操作的平均时间复杂度是:
 - A) $O(1)$
 - B) $O(\log n)$
 - C) $O(n)$
 - D) $O(n^2)$